

Simulado de Linguagens de Programação

Preparação para a Prova 1 – 2026.2

Nome: _____

Instruções Gerais

- As questões que envolvam implementação devem ser resolvidas utilizando Python.
- Utilize exclusivamente as folhas de resposta distribuídas com a prova para registrar suas respostas.
- Em cada folha de resposta, escreva seu nome completo no canto superior esquerdo. Folhas sem identificação não serão corrigidas.
- As respostas podem ser escritas a lápis ou a caneta, desde que estejam claramente legíveis.
- Indique claramente o número de cada questão respondida e, quando necessário, apresente os passos utilizados para chegar à solução.
- Ao final do simulado, grampeie todas as folhas de resposta junto ao simulado antes de entregá-lo.

Questão 1 (4,0 pontos)

Considere uma instituição fictícia que utiliza uma versão simplificada, para fins deste exercício, do cálculo do IRPF anual. Um contribuinte possui dois tipos de rendimento no ano:

- **rendimento tributável:** sujeito à tabela progressiva do IRPF;
- **rendimento não tributável** (ex.: poupança, alguns tipos de indenização): não entra na base de cálculo progressiva.

O imposto devido é calculado em duas etapas:

1. Imposto progressivo, aplicado apenas sobre o rendimento tributável, segundo a tabela abaixo:

Faixa de rendimento tributável (R\$)	Alíquota	Parcela a deduzir (R\$)
Até 24.000,00	Isento	0,00
De 24.000,01 até 36.000,00	15%	3.600,00
De 36.000,01 até 55.000,00	22%	6.120,00
Acima de 55.000,00	27%	8.870,00

O imposto progressivo é dado por:

$$\text{imposto} = (\text{rendimento_tributavel} \times \text{alíquota da faixa}) - \text{parcela a deduzir da faixa}$$

2. Alíquota mínima. Independentemente do resultado acima, a instituição exige que o imposto pago corresponda a, no mínimo, **10% do rendimento total do ano** (tributável + não tributável). Ou seja, mesmo um contribuinte com toda a renda não tributável deve pagar ao menos esse mínimo.

O imposto final é o **maior valor** entre o imposto progressivo e o imposto mínimo.

Por exemplo:

Tributável	Não tributável	Progressivo	Mínimo (10%)	Imposto final
20.000,00	0,00	0,00	2.000,00	2.000,00
40.000,00	0,00	2.680,00	4.000,00	4.000,00
0,00	50.000,00	0,00	5.000,00	5.000,00
100.000,00	0,00	18.130,00	10.000,00	18.130,00

Valores de rendimento negativos são inválidos e devem provocar uma exceção específica criada para representar esse erro de negócio. O resultado deve ser arredondado para 2 casas decimais.

- (a) Implemente a função `calcular_imposto(rend_tributavel, rend_ nao_tributavel)`, que retorna o imposto final devido.
- (b) Inclua comentários, docstring (Google ou NumPy), type hints e ao menos 2 testes com `doctest`.
- (c) Indique o resultado (ou a exceção produzida) para cada chamada abaixo:

```
1 calcular_imposto(30000, 5000)
2 calcular_imposto(0, 0)
3 calcular_imposto(-1000, 2000)
```

Questão 2 (6,0 pontos)

Um hospital organiza sua escala de plantão em três turnos fixos por dia: "manha", "tarde" e "noite". Cada turno precisa de uma quantidade mínima de pessoas escaladas, que pode variar por turno (por exemplo, a noite pode exigir menos pessoas que a manhã).

Além disso, **cada pessoa só pode ser escalada em um único turno por dia.**

A escala é representada por uma lista de tuplas (`essoa, dia, turno`). Como nem todo dia do período necessariamente possui algum plantão cadastrado, também é fornecida a lista `dias_validos` (strings), com todos os dias que deveriam ser cobertos pela escala – isso é necessário para que um dia sem nenhum plantão cadastrado também possa ser identificado como tendo turnos sem pessoal suficiente.

Considere a seguinte implementação, que deveria retornar quais (`dia, turno`) não atingiram o mínimo de pessoas exigido:

```
1 def verificar_escala(plantoess, dias_validos, minimos):
2     contagem = {}
3     pessoas_por_dia = {}
4
5     for pessoa, dia, turno in plantoes:
6         if dia not in contagem:
7             contagem[dia] = {"manha": 0, "tarde": 0, "noite": 0}
8             contagem[dia][turno] += 1
9
10        if dia not in pessoas_por_dia:
11            pessoas_por_dia[dia] = []
12        if pessoa in pessoas_por_dia[dia]:
13            print(f"Atencao: {pessoa} ja esta escalado em {dia}")
14        pessoas_por_dia[dia].append(pessoa)
15
16    faltando = {}
17    for dia in contagem:
18        for turno in contagem[dia]:
19            if contagem[dia][turno] < minimos[turno]:
20                if dia not in faltando:
21                    faltando[dia] = {}
22                    faltando[dia][turno] = {
23                        "atual": contagem[dia][turno],
24                        "minimo": minimos[turno]
25                    }
26
27    return faltando
```

Considere os parâmetros fixos abaixo para todos os itens desta questão:

```
1 dias_validos = ["01/09", "02/09", "03/09"]
2 minimos = {"manha": 2, "tarde": 2, "noite": 1}
```

- (a) Considerando a chamada abaixo, determine o valor exato retornado pela função `verificar_escala` tal como está implementada, apresentando os estados intermediários relevantes para justificar sua resposta:

```
1 plantoes = [
2     ("Ana", "01/09", "manha"),
3     ("Bruno", "01/09", "manha"),
4     ("Carla", "01/09", "tarde"),
5     ("Ana", "01/09", "tarde"),
6     ("Diego", "02/09", "noite"),
7 ]
8 verificar_escala(plantoes, dias_validos, minimos)
```

- (b) Para cada um dos casos de teste abaixo, indique se a função **passa** ou **falha** em relação ao resultado esperado (dado), justificando com base no comportamento real do código:

- **Teste 1:** mesma chamada do item (a). Resultado esperado: a saída deve reportar que o dia "03/09" está com **todos** os turnos abaixo do mínimo (nenhum plantão foi cadastrado nesse dia).
- **Teste 2:** mesma chamada do item (a). Resultado esperado: a função deve sinalizar programaticamente que Ana foi escalada duas vezes no dia "01/09", por meio de uma exceção ou de informação presente no valor retornado. Uma mensagem produzida apenas com "print" não satisfaz esse requisito.
- **Teste 3:**

```
1 plantoes = [  
2     ("Ana", "01/09", "manha"),  
3     ("Bruno", "01/09", "manha"),  
4     ("Carla", "01/09", "tarde"),  
5     ("Diego", "01/09", "tarde"),  
6     ("Elis", "01/09", "noite"),  
7 ]  
8 verificar_escala(plantoes, dias_validos, minimos)
```

Resultado esperado: dia "01/09" sem nenhuma pendência (todos os turnos atingem o mínimo); dias "02/09" e "03/09" com todos os turnos reportados como abaixo do mínimo.

- (c) Identifique os problemas existentes na implementação que explicam as falhas observadas no item anterior, e explique como cada um deveria ser corrigido.